

Diagnosing Neural-Network Training Behaviour from Loss Curves: A Controlled Keras Study of Dataset Size and the Three Hyperparameter Pillars

Mark Emmanuel P. Sagarino
College of Computer Studies
Course ITC-C508
Mandaluyong City, Philippines
markemmanuel.sagarino@my.jru.edu

Abstract—This paper reports a code-based Keras study of neural-network training dynamics for three-class text-intent classification. Seven experiments varied the three hyperparameter pillars across two datasets: the instructor-provided fifteen-row sample set (EXP-01 to EXP-05) and a 210-row food-delivery set (EXP-05A, EXP-05B). Every run was diagnosed from its loss curve. On the fifteen-row set no configuration achieved a good fit: a tiny learning rate produced underfitting at the chance-level loss, high capacity without regularization produced severe overfitting with a validation loss of 15.24, and dropout reduced but did not remove divergence. Accuracy on that set is also shown to be unreliable, because the split leaves only two validation samples; one underfitted run recorded a misleading 100 percent while its loss stayed at the $\ln(3)$ chance level. Only on the 210-row dataset did both curves converge, giving an ideal fit at 96.77 percent accuracy over thirty-one samples. Dataset volume, not tuning, was the binding constraint.

Keywords—neural network training; Keras; loss curves; underfitting; overfitting; regularization; dataset size; validation reliability.

I. INTRODUCTION AND PT-P1 BASELINE PERFORMANCE

Exercise PT-P1 built a text-intent classifier on a no-code platform that concealed every training detail behind automated heuristics, reporting 100 percent accuracy on ten hand-picked phrases. This paper re-implements the same three-class problem directly in Python with TensorFlow/Keras so that the loss function, learning rate, optimizer, and regularization become visible and controllable. That control makes it possible to do something a no-code tool cannot: deliberately drive a model into underfitting and into overfitting, and observe the distinct signature each leaves in the loss curves.

Seven experiments were executed. EXP-01 through EXP-05 train on the instructor-provided sample_dataset.csv, and EXP-05A and EXP-05B are free-form optimization runs on a larger food-delivery dataset. Each run is diagnosed from its loss curve, logged, and compared. A secondary finding, developed in Section IV-C, concerns the reliability of accuracy itself when the validation split is extremely small.

A. PT-P1 Baseline and the Benchmark Comparison

The PT-P1 no-code model recorded 100 percent accuracy and a weighted F1-score of 1.00, and this is the nominal benchmark the exercise asks the code-based model to exceed. The champion of the present study, EXP-05A, reached 96.77 percent validation accuracy and a weighted F1 of 0.9676, so on the raw numbers it does not surpass the benchmark. That comparison, however, is not like for like. The PT-P1 figure was obtained on ten phrases that were hand-picked after the

model was built, whereas 96.77 percent was measured on a held-out split of thirty-one samples the model never saw during training. A perfect score on ten curated examples is a far weaker claim than 96.8 percent on an unseen partition, and the honest conclusion is that the two numbers are not directly comparable rather than that one model is better. Where the code-based approach clearly wins is diagnostic: it exposes why a model succeeds or fails, which the no-code tool cannot do at all.

II. EXPERIMENTAL METHODOLOGY

A. Datasets and Preprocessing

Two datasets were used. The instructor-provided sample_dataset.csv contains fifteen course-feedback messages evenly split across three classes (Negative Feedback, Positive Feedback, Urgent Support) and drives EXP-01 to EXP-05. Our own PT-P1_food.csv contains 210 balanced food-delivery messages across three intent classes and drives the free-form runs EXP-05A and EXP-05B. Each message was lowercased, tokenized, mapped to integer identifiers against a capped vocabulary, and padded or truncated to a fixed length of twenty tokens. Index 0 was reserved for padding and index 1 for unknown words. Both datasets were partitioned 70/15/15 into training, validation, and test sets.

B. Model Architecture and the Three Pillars

The network is a Keras Sequential model consisting of an Embedding layer, a GlobalAveragePooling1D layer, a Dense ReLU hidden layer, a Dropout layer, and a softmax output, compiled with sparse categorical cross-entropy and the Adam optimizer. The Embedding and pooling layers constitute the automated feature-extraction stage. The tunable settings map onto the three pillars from Lecture 3: Architecture (embed_dim, hidden_dim), Optimization (learning_rate, batch_size, epochs, optimizer_type), and Regularization (dropout_rate). Early stopping with best-weight restoration was enabled for EXP-05, EXP-05A, and EXP-05B. Table I summarizes every configuration and its final measured results.

TABLE I. EXPERIMENT CONFIGURATIONS AND FINAL RESULTS

Run	Data	LR	Hid	Dr	Ep	Val n	Val Loss	Val Acc	Fit
EXP-01	sample	0.005	64	0.1	15	2	1.296	0.0%	Over
EXP-02	sample	0.0001	16	0.0	15	2	1.095	100%*	Under
EXP-03	sample	0.05	256	0.0	15	2	15.235	0.0%	Over
EXP-04	sample	0.005	128	0.4	15	2	1.210	0.0%	Over
EXP-05	sample	0.005	128	0.3	9	2	1.126	50.0%	Over
EXP-05A	food	0.003	96	0.3	20	31	0.066	96.8%	Good
EXP-05B	food	0.005	128	0.35	12	31	0.095	96.8%	Good

a. LR=learning rate, Hid=hidden units, Dr=dropout rate, Ep=epochs actually run, Val n=number of validation samples, Fit=loss-curve diagnosis. Champion: EXP-05A. *EXP-02 records 100%

accuracy on only two validation samples while its loss sits at the $\ln(3)=1.0986$ chance level; the figure is an artifact, not a result (Section IV-C).

III. PERFORMANCE MONITORING AND LOSS-CURVE ANALYSIS

Each experiment produced a training-versus-validation loss curve (Figures 1 to 7). On a dataset this small the loss curve, not the accuracy score, is the reliable diagnostic.

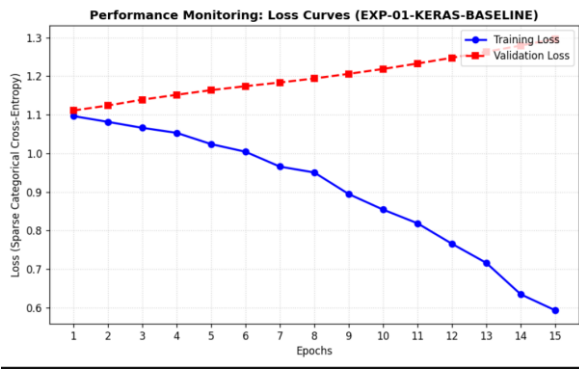


Fig. 1. EXP-01 baseline — training loss falls while validation loss rises (overfitting from data starvation).

EXP-01 (Fig. 1) shows the training loss descending steadily from 1.10 to 0.59 while the validation loss rises from 1.11 to 1.296. The two curves move in opposite directions from the very first epoch: with roughly ten training rows the model has enough capacity to memorize its examples immediately, and every additional epoch worsens generalization.

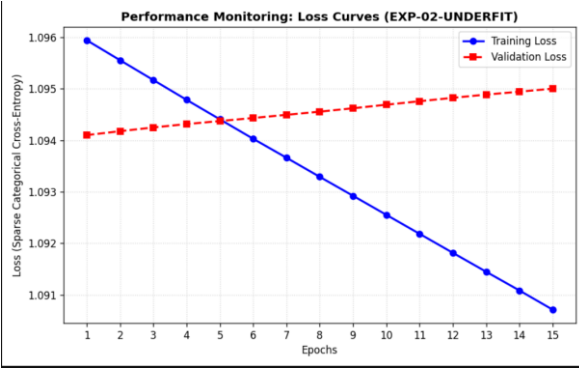


Fig. 2. EXP-02 — both losses remain at approximately 1.09, the $\ln(3)$ chance level (underfitting).

EXP-02 (Fig. 2) combines a learning rate of 0.0001 with only sixteen hidden units. Both curves stay near 1.09 across all fifteen epochs and change by less than 0.005 in total. Because $\ln(3) = 1.0986$ is the cross-entropy of a uniform guess over three classes, this run never departed from chance: the update steps were too small and the capacity too limited for any learning to occur.

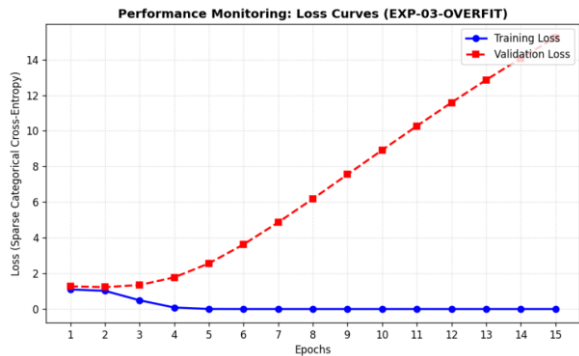


Fig. 3. EXP-03 — training loss reaches zero while validation loss diverges to 15.24 (severe overfitting).

EXP-03 (Fig. 3) applies high capacity (256 units) and a large learning rate (0.05) with no dropout. The training loss collapses to 0.0000 by the fourth epoch while the validation loss climbs without bound to 15.235, producing a generalization gap of 15.2. This is memorization in its most extreme form and the clearest overfitting signature in the study.

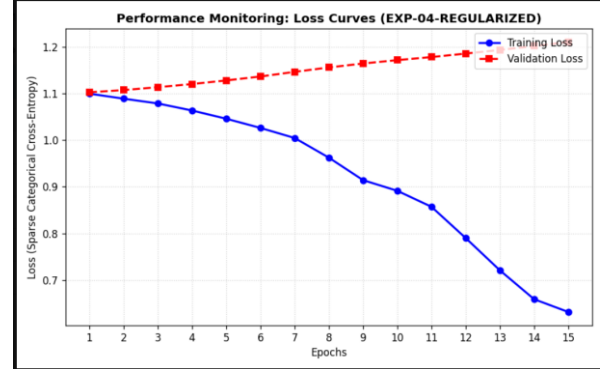


Fig. 4. EXP-04 — dropout reduces the divergence twelvefold, but validation loss still climbs.

EXP-04 (Fig. 4) retains the high capacity but adds dropout of 0.4. The effect is substantial: the final validation loss falls from 15.235 to 1.210, a twelvefold improvement over EXP-03. Yet the validation curve still rises rather than falls, showing that regularization mitigated the symptom without removing the cause.

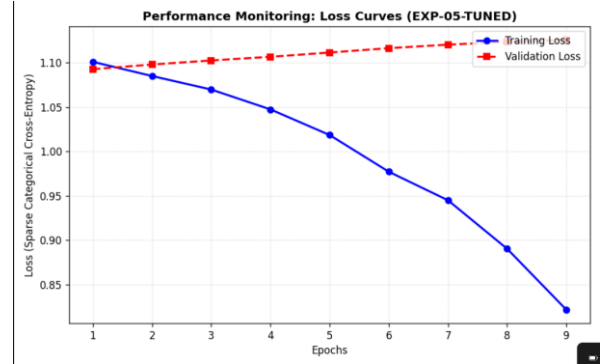


Fig. 5. EXP-05 — early stopping halts training at epoch 9; the curves still diverge.

EXP-05 (Fig. 5) is the tuned configuration with early stopping, which halted training at epoch 9 of a possible 60 once the validation loss stopped improving. It achieves the best validation loss on the sample data (1.126), but the curves continue to separate. No combination of settings tested here produced a good fit on fifteen rows.

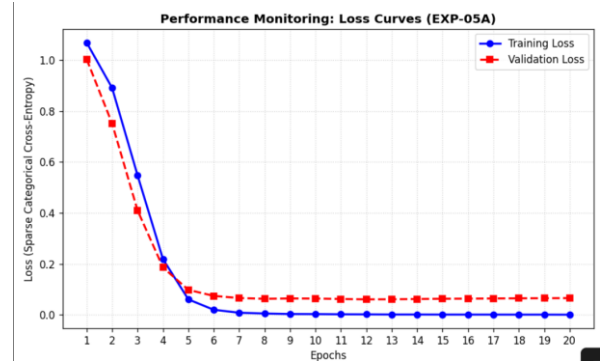


Fig. 6. EXP-05A champion — both losses fall together and flatten close to each other (ideal fit).

EXP-05A (Fig. 6) is the first run on the 210-row food dataset, and the change in behaviour is immediate. Both curves descend together, cross the 0.2 mark by the fourth epoch, and flatten with the validation loss settling at 0.0657 against a training loss of 0.0005. The small, stable gap is the textbook shape of a good fit, and validation accuracy reaches 96.77 percent. Nothing about the architecture family changed; only the quantity and diversity of the data did.

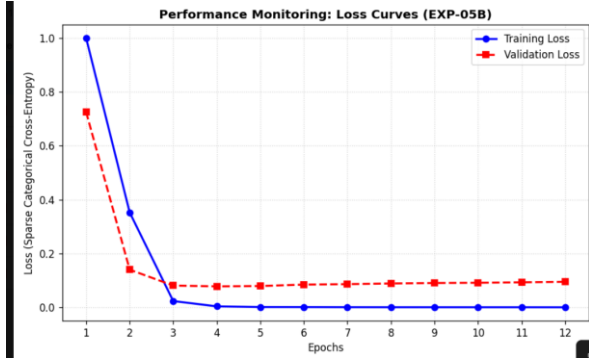


Fig. 7. EXP-05B — the same healthy convergence with a slightly larger gap, confirming stability.

EXP-05B (Fig. 7) uses a smaller batch size of eight and stronger dropout of 0.35 on the same dataset, early-stopping at epoch 12. It reproduces the same convergent shape and the identical 96.77 percent accuracy with a marginally higher validation loss of 0.0949, confirming that the good fit is a property of the dataset rather than of one lucky configuration.

IV. DISCUSSION AND DEBRIEF

A. Code-Based Optimization versus No-Code Heuristics

Writing the pipeline in Keras exposed four controls the PT-P1 no-code tool hid completely: the loss function, the learning rate, the optimizer, and the number of epochs. That visibility is what made this study possible, because inducing underfitting and overfitting on demand requires setting those values deliberately. The trade-off is effort and risk. The no-code tool produced a usable model instantly with no configuration, whereas the code-based approach required seven runs and careful reasoning, and a single poor choice — the 0.05 learning rate in EXP-03 — destroyed generalization entirely. Explicit control buys diagnostic insight; no-code automation buys speed and safety.

B. Diagnosing Fit via Loss Curves and Regularization

The two failure modes are distinguishable at a glance. Underfitting (EXP-02) appears as two high, flat, nearly parallel curves that never descend, pinned at the chance-level loss. Overfitting (EXP-03) appears as a training loss driven to zero while the validation loss turns upward and diverges. The pillars act in opposite directions. Raising hidden_dim from 16 to 256 supplied the capacity needed to fit the data, but that same capacity is precisely what enabled memorization once regularization was absent. Adding dropout of 0.4 in EXP-04 cut the final validation loss from 15.235 to 1.210, demonstrating that regularization directly counteracts the capacity it is paired with.

C. Dataset Size, and Why Accuracy Can Be Misleading

The most important finding of this study is that dataset size governed both the outcomes and the trustworthiness of the metrics. Because sample_dataset.csv holds only fifteen rows, a 70/15/15 partition leaves a validation set of just two samples. Accuracy measured on two samples can take only three values — 0, 50, or 100 percent — so it carries almost

no information. EXP-02 illustrates the danger concretely: it recorded a perfect 100 percent validation accuracy and an F1-score of 1.00 while its validation loss remained at 1.095, essentially the $\ln(3) = 1.0986$ chance level. A model that has demonstrably learned nothing can appear perfect if the validation set is small enough for chance to dominate. For this reason the champion was selected only from runs whose validation set contained at least ten samples, and the loss curves rather than the accuracy figures were used for diagnosis throughout Section III. On the 210-row dataset the validation set holds thirty-one samples, and there the 96.77 percent figure is meaningful. Hyperparameter tuning alone never produced a good fit on fifteen rows; expanding the data produced one immediately. Capacity must therefore be matched to data volume and diversity, and evaluation sets must be large enough for their metrics to mean anything.

D. Connecting Code Implementation to the Slide Architecture (Pipeline Mapping)

The training mathematics from Lecture 3 all take place inside the `model.fit(...)` call in CELL 6 of the notebook. For every batch of every epoch, Keras executes, in order: the Forward Pass, in which the input flows through Embedding to GlobalAveragePooling1D to Dense to Dropout to the softmax output; the Loss Calculation, using sparse categorical cross-entropy to compare predictions against the integer labels; the Backward Pass, in which backpropagation computes the gradient of that loss with respect to every weight; and the Weight Update, in which the Adam optimizer adjusts the parameters. The Embedding and GlobalAveragePooling1D layers together form the automated feature-extraction stage: the Embedding converts each token identifier into a trainable dense vector, and the pooling layer averages those vectors into a single fixed-length sentence representation that the Dense layers then classify. CELL 4 defines this graph, CELL 5 attaches the loss and optimizer, CELL 6 executes the four training steps, and CELL 7 plots the resulting loss history.

V. CONCLUSION

This study re-implemented a three-class text classifier as an explicit Keras pipeline and used seven controlled experiments to examine how the three hyperparameter pillars shape training. Every behaviour described in Lecture 3 was reproduced and identified from its loss curve: underfitting pinned at the chance-level loss, severe overfitting with a validation loss of 15.24, the corrective but partial effect of dropout, and an early-stopping point. The decisive result is that no configuration achieved a good fit on the fifteen-row dataset, while the very first run on the 210-row dataset converged cleanly to a validation loss of 0.0657 and 96.77 percent accuracy. A second finding is methodological: accuracy computed on a two-sample validation split is unreliable, as EXP-02 demonstrated by scoring a perfect 100 percent while its loss showed it had learned nothing. Both findings point the same way — data volume and diversity, together with evaluation sets large enough to be trusted, matter more than any amount of hyperparameter tuning.

REFERENCES

- [1] R. C. Raga Jr., "ITC 508 Lecture 3: Neural Network Training and Testing," course lecture, 2026.
- [2] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: <https://www.tensorflow.org>.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

- [4] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY, USA: Manning, 2021.